Проектирование реляционной базы данных

От предметной области интернет-магазина к таблицам, ключам и ограничениям

Результаты занятия

- ✓ Выделять сущности предметной области
- ✓ Определять атрибуты сущностей
- ✓ Преобразовывать сущности в таблицы PostgreSQL
- ✓ Выбирать подходящие типы данных
- ✓ Использовать PRIMARY KEY, NOT NULL, UNIQUE и CHECK
- ✓ Объяснять назначение FOREIGN KEY
- ✓ Уменьшать дублирование данных через разделение таблиц
- ✓ Проектировать базовую схему базы данных интернет-магазина

Учебные вопросы

1	**От предметной области к сущностям**	Что хранит приложение и какие объекты нужно представить в базе данных
2	**Таблицы, столбцы и типы данных**	Как представить свойства сущности средствами PostgreSQL
3	**Ключи и ограничения**	PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE и CHECK
4	**Устранение дублирования**	Почему повторяющиеся данные следует выносить в отдельные таблицы
5	**Проектирование схемы Internet Shop**	Пошаговый алгоритм создания структуры базы данных

Учебный вопрос 1. От предметной области к сущностям

Сначала определяем, какие данные нужны приложению, и только потом создаём таблицы

Что такое предметная область

- Предметная область — часть реального мира, информацию о которой должно хранить и обрабатывать приложение.
- Для интернет-магазина предметная область включает товары, категории, покупателей и заказы.
- Каждый объект имеет собственные свойства.
- Некоторые объекты связаны друг с другом.
- Проектирование базы начинается с анализа этих объектов и правил работы приложения.
- Названия таблиц не следует придумывать раньше, чем понятен смысл хранимых данных.

Что такое сущность

- Сущность — тип объекта предметной области, экземпляры которого необходимо хранить в базе данных.
- Product — товар интернет-магазина.
- Category — категория товаров.
- User — пользователь или покупатель.
- Order — заказ.
- Каждая сущность обычно становится отдельной таблицей.

Сущности Internet Shop

Product

Товар: название, цена, наличие и категория.

Category

Категория, к которой относится товар.

User

Пользователь приложения: имя, email и другие данные.

Order

Заказ, созданный пользователем.

Order Item

Позиция заказа: товар, количество и цена.

От предметной области к сущностям базы данных

Сначала определяем объекты реального мира, затем проектируем таблицы

Интернет-магазин как пример

Предметная область: Интернет-магазин

Реальные объекты и процессы, с которыми работает бизнес



Товары
Ноутбуки, мыши, наушники и другие продукты

- Laptops
- Monitors
- Accessories

Категории
Группы товаров по назначению



Покупатель
Регистрируется, выбирает товары, оформляет заказ



Корзина и заказ
Товары в корзине, оформление и оплата

Анализ предметной области

1. **Выделяем объекты (сущности)**
   Какие данные нужно хранить?
2. **Определяем их свойства**
   Какие атрибуты у каждого объекта?
3. **Находим связи между объектами**
   Как они связаны друг с другом?
4. **Преобразуем в таблицы**
   Создаём структуру базы данных PostgreSQL.

Сущности базы данных

Основные объекты, которые будем хранить в PostgreSQL

Product
- id
- name
- price
- in_stock
- category_id

Товар интернет-магазина

Category
- id
- name

Категория товаров

User
- id
- name
- email

Пользователь (покупатель)

Order
- id
- user_id
- status
- created_at

Заказ, созданный пользователем

Order Item
- order_id
- product_id
- quantity
- price

Позиция заказа (товар и количество)

Сначала понимаем бизнес
Предметная область → объекты → атрибуты → связи

Затем проектируем таблицы
Сущности → таблицы → столбцы → ограничения

Что такое атрибут

- Атрибут — характеристика сущности, значение которой необходимо хранить.
- Для Product атрибутами могут быть name, price и inStock.
- Для Category нужен атрибут name.
- Для User могут использоваться name и email.
- Каждый атрибут обычно становится столбцом таблицы.
- Не следует добавлять столбец, если приложение не понимает его назначение.

Сущности и их атрибуты

Сущность	Возможные атрибуты
Product	id, name, price, in_stock, category_id
Category	id, name
User	id, name, email
Order	id, user_id, status, created_at
Order Item	order_id, product_id, quantity, price

Сущность и атрибут — не одно и то же

Сущность

- Самостоятельный объект предметной области
- Имеет собственные экземпляры
- Обычно становится таблицей
- Пример: Category

Атрибут

- Свойство сущности
- Описывает один экземпляр
- Обычно становится столбцом
- Пример: category.name

Учебный вопрос 2. Таблицы, столбцы и типы данных

Переводим модель предметной области в структуру PostgreSQL

Сущность превращается в таблицу

- Таблица хранит множество экземпляров одной сущности.
- Каждая строка представляет один объект.
- Каждый столбец представляет один атрибут.
- Структура таблицы определяется командой CREATE TABLE.
- Для каждого столбца необходимо выбрать имя и тип данных.

Сущность → таблица → строка → столбец

От понятия предметной области к конкретным данным в базе

Пример: интернет-магазин

1 Сущность (Entity)

Тип объекта предметной области, экземпляры которого мы храним в базе данных.



Product

Товар интернет-магазина

Сущность = тип объекта

«Какие объекты мы храним?»

2 Таблица (Table)

Таблица хранит множество экземпляров одной сущности. Каждый столбец — это атрибут сущности.

products

id	name	price	category_id	in_stock
1	Keyboard	15000	1	true
2	Mouse	8000	1	true
3	Monitor	85000	2	false
...	...	...	...	...

Таблица = множество объектов одного типа

«Где хранятся эти объекты?»

3 Строка (Row)

Строка представляет один конкретный экземпляр сущности. Это отдельный товар.

Одна строка таблицы products

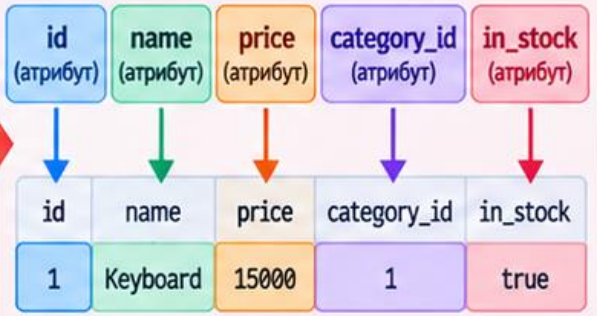
id	name	price	category_id	in_stock
1	Keyboard	15000	1	true

Строка = экземпляр сущности

«Один конкретный объект»

4 Столбец (Column)

Столбец хранит одно свойство (атрибут) сущности.
У каждой строки есть значение в каждом столбце.



Столбец = атрибут сущности

«Какое свойство объекта?»

Главная идея

Сущность описывает, какие объекты мы храним. Таблица хранит эти объекты. Строка — один конкретный объект. Столбец — его свойство.

Типы данных для Internet Shop

Тип PostgreSQL	Что хранить	Пример
INTEGER	Целые числа, id, цена в целых тенге	45000
VARCHAR(n)	Строки ограниченной длины	Keyboard
TEXT	Текст без заранее заданной длины	Описание товара
BOOLEAN	Логические значения	true
TIMESTAMPTZ	Дата и время события	Время создания заказа

Как выбирать тип данных

- Тип должен соответствовать смыслу значения.
- id удобно хранить как INTEGER identity.
- Логическое состояние следует хранить BOOLEAN, а не строками yes/no.
- Цена в текущем учебном проекте хранится как INTEGER в тенге.
- Если приложение требует дробные денежные значения, можно использовать NUMERIC.
- Дата и время должны храниться специальным типом, а не произвольной строкой.

Удачный и неудачный выбор типов

Неудачно

- price VARCHAR
- in_stock VARCHAR
- created_at VARCHAR
- Все данные хранятся как строки
- Сложнее проверять и обрабатывать значения

Лучше

- price INTEGER
- in_stock BOOLEAN
- created_at TIMESTAMPTZ
- Тип отражает смысл значения
- PostgreSQL помогает контролировать данные

Соглашения об именовании

- Имена таблиц и столбцов должны быть понятными и последовательными.
- В PostgreSQL удобно использовать нижний регистр и snake_case.
- Например: category_id, in_stock, created_at.
- Одинаковые понятия должны называться одинаково во всей схеме.
- Не следует использовать непонятные сокращения вроде catid или prodnm.

Первая таблица categories

```sql
CREATE TABLE categories (
  id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name VARCHAR(100) NOT NULL UNIQUE
);
```

Что важно

- Каждая категория получает собственный id.
- name обязательно должно быть заполнено.
- UNIQUE запрещает две категории с одинаковым названием.
- Таблица хранит категории независимо от товаров.

Учебный вопрос 3. Ключи и ограничения

База данных должна сама защищать важные правила целостности

Первичный ключ

- PRIMARY KEY — столбец или набор столбцов, однозначно идентифицирующий каждую строку таблицы.
- Значение первичного ключа должно быть уникальным.
- PRIMARY KEY не допускает NULL.
- Через id приложение может обратиться к конкретной строке.
- Другие таблицы могут ссылаться на первичный ключ.
- В нашем проекте большинство таблиц получает числовой id.

Естественный и искусственный идентификатор

Естественный ключ

- Использует существующее бизнес-значение
- Например, email
- Значение может измениться
- Может быть длинным

Искусственный id

- Создаётся специально для идентификации
- Не зависит от бизнес-значения
- Удобен для внешних ключей
- В проекте используем INTEGER identity

Основные ограничения

Ограничение	Что гарантирует
PRIMARY KEY	Строка имеет уникальный идентификатор
NOT NULL	Значение обязательно присутствует
UNIQUE	Значение или комбинация значений не повторяется
CHECK	Значение соответствует заданному условию
FOREIGN KEY	Ссылка указывает на существующую строку другой таблицы

Ограничения защищают Product

```sql
CREATE TABLE products (
  id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name VARCHAR(150) NOT NULL,
  price INTEGER NOT NULL CHECK (price > 0),
  in_stock BOOLEAN NOT NULL DEFAULT true
);
```

Что важно

- PRIMARY KEY гарантирует уникальный id.
- NOT NULL запрещает отсутствие name и price.
- CHECK не позволяет сохранить неположительную цену.
- DEFAULT true задаёт значение наличия по умолчанию.
- Правила действуют независимо от того, какое приложение выполняет INSERT.

Что такое внешний ключ

- FOREIGN KEY — ограничение, которое связывает значение столбца одной таблицы с существующей строкой другой таблицы.
- В products можно хранить category_id.
- category_id должен ссылаться на существующую category.id.
- PostgreSQL не позволит сослаться на несуществующую категорию.
- Внешний ключ поддерживает ссылочную целостность.
- Подробно типы связей между таблицами рассмотрим в следующей лекции.

Связываем Product с Category

```sql
CREATE TABLE products (
  id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name VARCHAR(150) NOT NULL,
  price INTEGER NOT NULL CHECK (price > 0),
  category_id INTEGER NOT NULL REFERENCES categories(id),
  in_stock BOOLEAN NOT NULL DEFAULT true
);
```

Что важно

- category_id хранит числовой идентификатор категории.
- REFERENCES categories(id) создаёт внешний ключ.
- Каждый ненулевой category_id должен существовать в categories.
- Название категории больше не требуется повторять в каждой строке products.

PRIMARY KEY и FOREIGN KEY связывают таблицы

Внешний ключ (FOREIGN KEY) обеспечивает ссылочную целостность: значение в одной таблице должно соответствовать существующей строке в другой таблице.

PostgreSQL

categories

Справочник категорий товаров

id PK	name
1	Accessories
2	Monitors
3	Components
4	Software
...	...

id — первичный ключ (PRIMARY KEY)
Уникально идентифицирует категорию.

products.category_id → categories.id

Связь: многие товары относятся к одной категории

products

Товары интернет-магазина

id PK	name	price	category_id FK	in_stock
1	Keyboard	15000	1	true
2	Mouse	8000	1	true
3	Monitor	85000	2	true
4	SSD 2TB	65000	3	true
...	...	...	...	...

id — первичный ключ (PRIMARY KEY)
Уникально идентифицирует товар.

category_id — внешний ключ (FOREIGN KEY)
Должен соответствовать существующему categories.id.

Корректная ссылка (категория существует)

category_id = 2 — такая категория есть в таблице categories.

```
INSERT INTO products (name, price, category_id, in_stock)
VALUES ('Monitor', 85000, 2, true);
```

Запрос выполнен успешно. Товар добавлен.

Ошибка: ссылка на несуществующую категорию

category_id = 999 — такой категории нет в таблице categories.

```
INSERT INTO products (name, price, category_id, in_stock)
VALUES ('Unknown Product', 12000, 999, true);
```

ERROR: insert or update on table products violates foreign key constraint "products_category_id_fkey"
DETAIL: Key (category_id)=(999) is not present in table "categories".

Что это даёт?

- Невозможно добавить товар с несуществующей категорией.
- Предотвращаются ошибки в данных.
- Сохраняется ссылочная целостность между таблицами.
- База данных сама контролирует корректность связей.

PostgreSQL защищает ссылочную целостность

Запрос: Добавить товар с category_id = 999

Код • SQL

```
INSERT INTO products
  (name, price, category_id, in_stock)
VALUES
  ('SSD 2TB', 65000, 999, true);
```

Результат • TEXT

```
ERROR: insert or update on
table products violates
foreign key constraint
```

Пояснение

- Если категории 999 нет в categories, PostgreSQL отклоняет вставку и не создаёт некорректную ссылку.

Где должны проверяться правила

- Frontend может проверять форму для удобства пользователя.
- Validation middleware проверяет HTTP-запрос до Controller.
- Service выполняет бизнес-операции.
- PostgreSQL дополнительно защищает данные ограничениями таблицы.
- Критичные правила целостности нельзя оставлять только на уровне frontend.

Главная идея

Валидация приложения и ограничения базы дополняют друг друга, а не заменяют друг друга.

Учебный вопрос 4. Устранение дублирования

Повторяющиеся данные приводят к несогласованности и усложняют изменения

Проблема текущей таблицы products

id	name	price	category
1	Keyboard	15000	Accessories
2	Mouse	8000	Accessories
3	Web Camera	22000	Accessories
4	Monitor	85000	Monitors

Почему повторение category неудобно

- Название Accessories повторяется во многих строках.
- Можно случайно получить Accessories, Accessory и accessories.
- Переименование категории потребует изменения множества товаров.
- Нельзя удобно хранить дополнительные свойства категории.
- Одни и те же данные занимают место во многих строках.

Категория строкой и отдельная таблица

category VARCHAR

- Название повторяется
- Возможны разные варианты написания
- Сложнее переименовать
- Нет единого объекта Category
- Дополнительные свойства хранить неудобно

category_id

- Категория хранится один раз
- Товар хранит только ссылку
- Название меняется в одном месте
- PostgreSQL проверяет ссылку
- Category можно расширять новыми полями

Как разделить Product и Category

1	**Выделить сущность Category**	Категория является самостоятельным объектом и получает таблицу categories.
2	**Создать category.id**	Каждая категория получает уникальный первичный ключ.
3	**Хранить название в categories**	Название категории больше не повторяется в каждой строке products.
4	**Добавить products.category_id**	Product хранит идентификатор категории вместо её названия.
5	**Создать FOREIGN KEY**	category_id связывается с categories.id и контролируется PostgreSQL.

Сегодня лучше, чем вчера в IT :)

От дублирования category к таблице categories

Убираем повторяющиеся данные. Делаем базу данных более удобной и надёжной.

Лучшие базы данных для больших возможностей!

Было: одна таблица с дублированием

products

id	name	price	category
1	Keyboard	2990	Accessories
2	Mouse	1590	Accessories
3	Web Camera	4990	Accessories
4	Monitor	18990	Monitors

Проблема:
category повторяется во многих строках.

Выносим категории в отдельную таблицу

Разделение

Accessories
Accessories
Accessories

Выделяем уникальные значения

Стало: нормализованная структура

categories

id (PK)	name
1	Accessories
2	Monitors
3	Components

Название категории хранится в одном месте

Изменили название здесь — и оно обновится для всех товаров этой категории.

products

id	name	price	category_id (FK)
1	Keyboard	2990	1
2	Mouse	1590	1
3	Web Camera	4990	1
4	Monitor	18990	2

products.category_id ссылается на categories.id

Преимущества такого подхода:

Нет дублирования
Каждая категория хранится только один раз.

Категорию легче переименовать
Достаточно изменить в одном месте.

Снижается риск ошибок
Нет противоречивых или несовпадающих названий.

База данных становится более согласованной
Упрощается поддержка и дальнейшее развитие.

Хорошая структура данных — фундамент надёжных приложений ♡

Больше, чем просто знания →

Что такое нормализация в практическом смысле

- Нормализация — организация структуры таблиц так, чтобы уменьшить лишнее дублирование и зависимости данных.
- Каждая таблица должна описывать понятную сущность.
- Одно значение не следует без необходимости повторять во множестве строк.
- Данные одной сущности желательно хранить в одном месте.
- Связанные сущности соединяются ключами.
- Цель — уменьшить вероятность противоречивых данных и упростить изменение информации.

Учебный вопрос 5. Проектирование схемы Internet Shop

Проектируем структуру до реализации новых REST endpoints

Базовые таблицы Internet Shop

categories

Справочник категорий товаров.

products

Каталог товаров со ссылкой на категорию.

users

Пользователи и покупатели интернет-магазина.

orders

Заказы пользователей.

order_items

Конкретные товары и количество внутри заказа.

Предварительная схема базы данных

Таблица	Основные столбцы
categories	id, name
products	id, name, price, category_id, in_stock
users	id, name, email
orders	id, user_id, status, created_at
order_items	order_id, product_id, quantity, price

Схема БД интернет-магазина

categories

Поле	Тип данных	Ключ
id	INT	PK
name	VARCHAR(100)	NOT NULL
description	TEXT	

products

Поле	Тип данных	Ключ
id	INT	PK
name	VARCHAR(150)	NOT NULL
price	INTEGER	NOT NULL
category_id	INT	FK
in_stock	BOOLEAN	NOT NULL

users

Поле	Тип данных	Ключ
id	INT	PK
name	VARCHAR(100)	NOT NULL
email	VARCHAR(100)	NOT NULL UNIQUE
password_hash	VARCHAR(255)	NOT NULL
created_at	TIMESTAMP	DEFAULT NOW()

orders

Поле	Тип данных	Ключ
id	INT	PK
user_id	INT	FK
order_date	TIMESTAMP	DEFAULT NOW()
status	VARCHAR(20)	NOT NULL
total_amount	INTEGER	NOT NULL

order_items

Поле	Тип данных	Ключ
id	INT	PK
order_id	INT	FK
product_id	INT	FK
quantity	INTEGER	NOT NULL
price	INTEGER	NOT NULL

1 — 0..N (categories → products)

1 — 0..N (products → order_items)

1 — 0..N (users → orders)

1 — 0..N (orders → order_items)

Условные обозначения

- PK – первичный ключ
- FK – внешний ключ
- Связь один ко многим (1 : 0..N)
- Таблица (имя, поля и типы данных)

Как работает база данных интернет-магазина

Корзина, оформление заказа и связи между таблицами

Пять таблиц базы данных

Таблица	Что хранит
categories	Категории товаров
products	Товары, цены и признак наличия
users	Данные зарегистрированных покупателей
orders	Заказы: покупатель, дата, статус, сумма
order_items	Позиции заказа: товар, количество, цена

Внешние ключи и отношения

Таблица-источник	Внешний ключ	Ссылка	Кардинальность
products	category_id	categories.id	1 категория → много товаров
orders	user_id	users.id	1 пользователь → много заказов
order_items	order_id	orders.id	1 заказ → много позиций
order_items	product_id	products.id	1 товар → много позиций заказов

Что происходит при добавлении в корзину

- Корзина — временный список товаров, которые покупатель собирается заказать.
- Покупатель выбирает товар и его количество.
- Клиент добавляет позицию в корзину и показывает предварительную сумму.
- В данной схеме отдельной таблицы корзины нет: данные можно хранить в localStorage или серверной сессии.
- Записи orders и order_items на этом этапе ещё не создаются.

Главная идея

При оформлении нельзя доверять цене, присланной клиентом: сервер должен получить актуальную цену из базы данных.

Пример корзины до оформления заказа

Товар	Количество	Цена за единицу	Сумма
Ноутбук	1	300 000 ₸	300 000 ₸
Мышь	2	8 000 ₸	16 000 ₸
ИТОГО			316 000 ₸

Оформление заказа: последовательность

1. **Шаг 1** — Клиент отправляет состав корзины на сервер.
2. **Шаг 2** — Сервер проверяет товары, цены и возможность продажи.
3. **Шаг 3** — В одной транзакции сервер создаёт запись orders.
4. **Шаг 4** — Затем добавляет позиции в order_items, связывая их с id заказа.
5. **Шаг 5** — При успехе выполняет COMMIT и возвращает клиенту номер заказа; при ошибке — ROLLBACK.

Запись в orders

id	user_id	status	total_amount
101	5	new	316000

Записи в order_items

id	order_id	product_id	quantity	price
1	101	10	1	300000
2	101	15	2	8000

Как рассчитывается сумма заказа

- Итоговая стоимость = сумма по всем позициям (quantity × price).
- Ноутбук: 1 × 300 000 = 300 000 ₸.
- Мышь: 2 × 8 000 = 16 000 ₸.
- Общая сумма: 300 000 + 16 000 = 316 000 ₸.
- Поле order_items.price фиксирует цену единицы товара на момент оформления.
- orders.total_amount сохраняет рассчитанный итог заказа.

Корзина и оформленный заказ

Корзина

- Временные данные до оформления.
- В этой модели может храниться на клиенте или в сессии.
- Содержимое можно менять без записи в orders.

Заказ

- Постоянные записи в базе данных.
- orders содержит сведения о заказе.
- order_items содержит товары и количество.
- Общие изменения оформляются транзакцией.

Ограничение упрощённой модели

- products.in_stock показывает только наличие или отсутствие товара.
- Это поле не содержит количество единиц на складе.
- Для проверки точного остатка нужна дополнительная модель складского учёта.
- При конкурентных заказах необходима согласованная проверка и изменение остатков внутри транзакции.

Главная идея

Схема из пяти таблиц демонстрирует оформление заказов, но не реализует полноценный складской учёт.

Алгоритм проектирования таблицы

1	**Определить сущность**	Понять, какой самостоятельный объект нужно хранить.
2	**Определить атрибуты**	Записать свойства объекта, необходимые приложению.
3	**Выбрать первичный ключ**	Определить способ однозначной идентификации строки.
4	**Выбрать типы данных**	Для каждого столбца выбрать подходящий тип PostgreSQL.
5	**Добавить ограничения**	Определить NOT NULL, UNIQUE, CHECK и внешние ключи.
6	**Проверить дублирование**	Убедиться, что данные разных сущностей не смешаны в одной таблице без необходимости.

Алгоритм проектирования реляционной таблицы

1 Предметная область



Например, интернет-магазин

2 Сущность



Например, «Товар»

3 Атрибуты



Определяем поля таблицы

4 PRIMARY KEY



Выбираем первичный ключ

5 Типы данных



Назначаем типы для атрибутов

6 Ограничения



NOT NULL
UNIQUE
CHECK
...

7 Внешние ключи



Связываем с другими таблицами

8 Проверка дублирования



Ищем и устраняем дубликаты (нормализация)

9 Готовая таблица PostgreSQL



Таблица создана и готова к использованию



Сначала модель данных — потом CREATE TABLE.



Целевая таблица categories

```sql
CREATE TABLE categories (
  id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name VARCHAR(100) NOT NULL UNIQUE
);
```

Что важно

- Category стала самостоятельной сущностью.
- id используется как стабильный идентификатор.
- name обязательно и уникально.
- В дальнейшем products будет ссылаться на categories.id.

Целевая таблица products

```sql
CREATE TABLE products (
  id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name VARCHAR(150) NOT NULL,
  price INTEGER NOT NULL CHECK (price > 0),
  category_id INTEGER NOT NULL REFERENCES categories(id),
  in_stock BOOLEAN NOT NULL DEFAULT true
);
```

Что важно

- Product имеет собственный первичный ключ.
- Цена защищена CHECK.
- category_id заменяет текстовый столбец category.
- FOREIGN KEY предотвращает ссылку на отсутствующую категорию.
- Структура готова к дальнейшему изучению связей между таблицами.

Схема базы и REST API — разные уровни

- Структура таблицы не обязана полностью совпадать со структурой JSON.
- В базе может использоваться category_id.
- REST API может возвращать клиенту categoryId и название категории.
- Service преобразует результат SQL в данные, необходимые Controller.
- Frontend не должен знать внутренние детали хранения данных.

Проверка качества схемы

Вопрос	Что проверить
Что хранит таблица?	Одна понятная сущность
Как определяется строка?	Есть PRIMARY KEY
Какие поля обязательны?	Использован NOT NULL
Что не должно повторяться?	Использован UNIQUE
Какие значения недопустимы?	Использован CHECK
Есть ссылка на другую сущность?	Использован FOREIGN KEY
Есть лишнее дублирование?	Повторяющиеся сущности вынесены отдельно

Мини-практика: спроектируйте Category и Product

Переработайте текущую структуру Internet Shop так, чтобы категории хранились отдельно от товаров.

Задание

- Определить атрибуты Category
- Создать таблицу categories
- Добавить PRIMARY KEY
- Добавить UNIQUE для названия категории
- Определить category_id в Product
- Создать FOREIGN KEY на categories(id)
- Перенести существующие категории из products
- Проверить корректность ссылок

Критерии проверки

- Категория является отдельной таблицей
- Название категории не дублируется в products
- Каждый товар связан с существующей категорией
- Некорректный category_id отклоняется PostgreSQL
- Исходные товары сохраняются после переработки схемы

Контрольные вопросы

1. Что такое предметная область?
2. Чем сущность отличается от атрибута?
3. Как сущность представляется в реляционной базе?
4. Зачем каждому столбцу задаётся тип данных?
5. Что гарантирует PRIMARY KEY?
6. Чем UNIQUE отличается от PRIMARY KEY?
7. Для чего используется CHECK?
8. Что проверяет FOREIGN KEY?
9. Почему хранить название категории в каждой строке products неудобно?
10. Какие шаги нужно выполнить перед созданием новой таблицы?

Итоги занятия

1. Проектирование базы начинается с анализа предметной области.
2. Самостоятельные объекты становятся сущностями и таблицами.
3. Свойства сущностей становятся столбцами.
4. Для каждого столбца выбирается тип данных, соответствующий смыслу значения.
5. PRIMARY KEY однозначно идентифицирует строки.
6. NOT NULL, UNIQUE и CHECK защищают правила целостности.
7. FOREIGN KEY поддерживает корректные ссылки между таблицами.
8. Повторяющиеся данные следует анализировать и при необходимости выносить в отдельные сущности.
9. Хорошая схема базы упрощает дальнейшую разработку REST API.